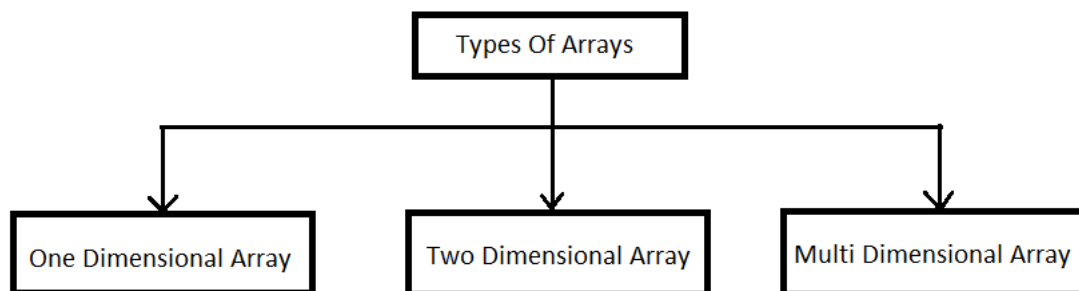


Java

Types of Array :-

- In Java, an array is a collection of elements of the same data type.
- It stores data in contiguous memory locations.
- Each element of an array is accessed by its index, starting from 0.
- Arrays make it easier to store and manage multiple values in a single variable.
- In Java, arrays are mainly of three types:



1. One Dimensional Array (1D) :-

- It stores elements in a linear form.
- Index starts from 0 and goes up to n-1.
- Useful for storing list-like data.

Syntax:

`datatype arrayName[] = { value 1, value 2, value 3,...}`

Example –

```
package Basic_Concept;
```

```
public class arraydemo {
```

```
    public static void main(String[] args) {
```

```
        int arr[] = {10,20,30,40,50};
```

```
        System.out.println("Length of Array is:"+arr.length);
```

```
        for(int i=0;i<arr.length;i++)  
        {  
            System.out.println(arr[i]);  
        }  
    }  
}
```

Output –

Length of Array is:5

10

20

30

40

50

2. Two Dimensional Array (2D) :-

- Array of arrays arranged in rows and columns.
- Commonly used for matrices and tables.

Syntax –

```
datatype arrayName[][] = { { value 1, value 2 },  
                             { Value 3, Value 4 } };
```

Example –

```
package Basic_Concept;
```

```
public class array2d {  
  
    public static void main(String[] args) {  
        int arr[][]= { {10,20},  
                        {30,40} };  
  
        for(int r=0;r<arr.length;r++)  
        {  
            for(int c=0;c<arr.length;c++)  
            {  
                System.out.print(arr[r][c]+" ");  
            }  
            System.out.println();  
        }  
    }  
}
```

Output –

```
10 20
```

```
30 40
```

3. Multi-Dimensional Array :-

- It is a special type of multidimensional array.
- Each row can have a different number of columns.
- Saves memory when rows have unequal elements.

Syntax –

```
datatype arrayName[][] = {  
    { { value1, value2 },  
      { value3, value4 }  
    },  
    { { value5, value6 },  
      { value7, value8 }  
    }  
};
```

Example –

```
package Basic_Concept;
```

```
public class multidimensional_array {  
  
    public static void main(String[] args) {  
        int arr[][][] = {  
            { {8, 7}, {6, 5} },  
            { {5, 3}, {2, 1} }  
        };  
  
        for (int i= 0;i< arr.length;i++)  
        {  
            for (int r=0;r< arr[i].length;r++)  
            {  
                for (int c=0;c<arr[i][r].length;c++)
```

```
        {  
            System.out.print(arr[i][r][c] + " ");  
        }  
        System.out.println();  
    }  
    System.out.println();  
}  
}
```

Output –

8 7

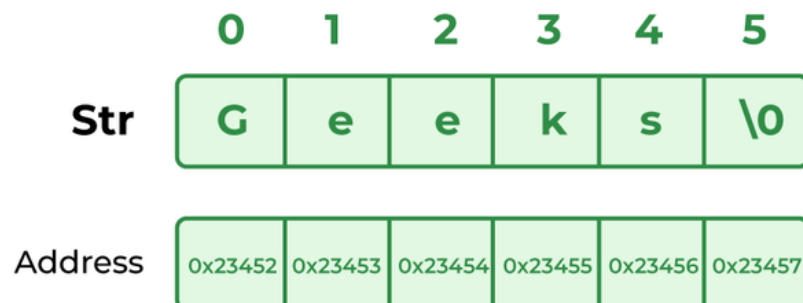
6 5

5 3

2 1

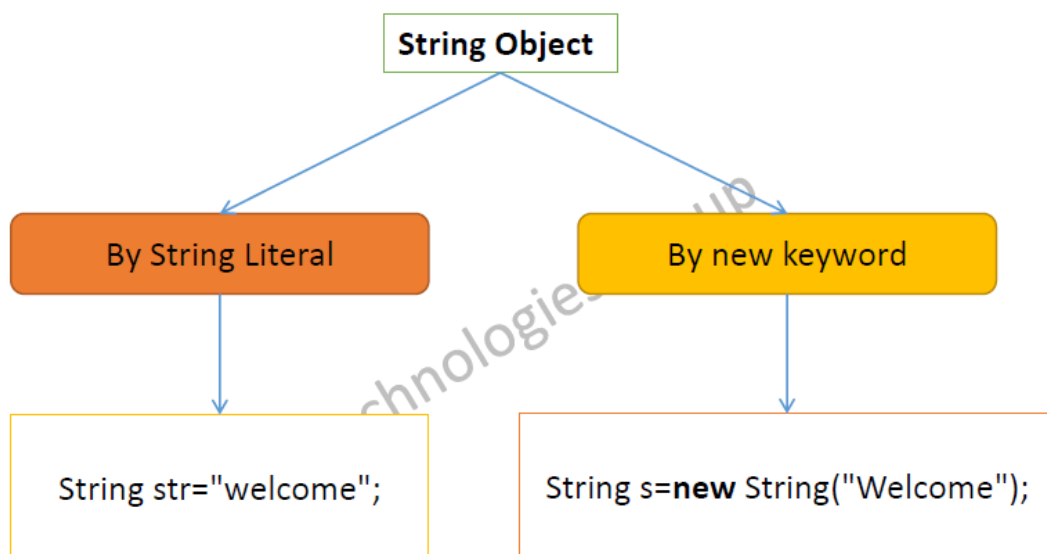
String in Java :-

- In Java, a String is a sequence of characters enclosed in double quotes.
- It is a class in java.lang package and is treated as an object.
- Strings are immutable, meaning their values cannot be changed after creation.



Declaration of String :-

- Strings can be declared using string literals & new keyword.



1) By String Literal –

- A string literal is enclosed in double quotes.
- It is stored in the String Constant Pool (SCP) for memory efficiency.
- If another string with the same value is created, it refers to the same object in SCP.

Syntax –

String str = "Sagar";

Example –

```
package Basic_Concept;

public class stringdemo {

    public static void main(String[] args) {

        String name="FortuneCloud";

        System.out.println("String is: "+name);

    }

}
```

Output –

String is: FortuneCloud

2) By new Keyword –

- Using the new keyword, a new String object is created in the heap memory.
- Even if the same string exists in SCP, a new object is always created.

Syntax –

```
String str = new String("Sagar");
```

Example –

```
package Basic_Concept;
```

```
public class stringdemo {  
  
    public static void main(String[] args) {  
  
        String s1 = new String("Sagar");  
        String s2 = new String("Kate");  
        System.out.println(s1);  
        System.out.println(s2);  
    }  
}
```

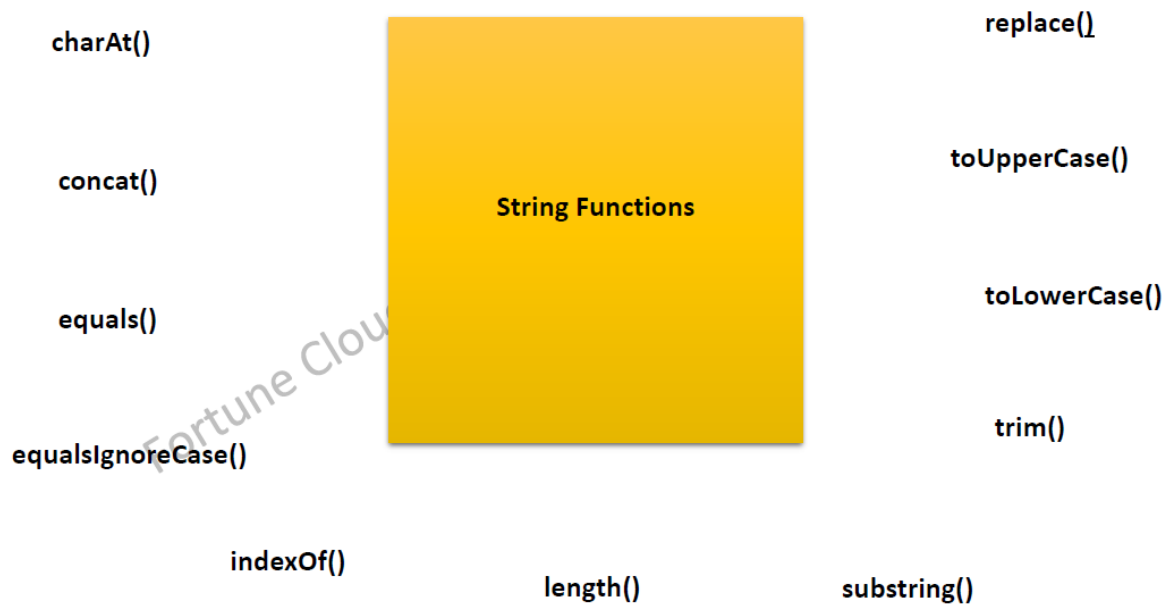
Output –

Sagar

Kate

String Functions :-

- The String class provides methods to manipulate and process strings.
- Strings are immutable, so most methods return a new string without changing the original.
- These functions help in comparison, case conversion, splitting, and extracting parts of a string.



1. **length()** - returns length of string.

Syntax –

```
str.length();
```

2. **charAt(int index)** - returns character at given index.

Syntax –

```
str.charAt(0);
```

3. **concat(String str)** - joins two strings.

Syntax –

```
str1.concat(str2);
```

4. **equals(Object obj)** - compares two strings (case-sensitive)

Syntax –

```
str1.equals(str2);
```

5. **toUpperCase()** - Converts the string to uppercase.

Syntax –

```
str.toUpperCase();
```

6. **toLowerCase()** - Converts the string to lowercase.

Syntax –

```
str.toLowerCase();
```

7. **startsWith(String prefix)** - Checks if string starts with the given prefix.

Syntax –

```
str.startsWith("Hi");
```

8. **endsWith(String suffix)** - Checks if string ends with the given suffix.

Syntax –

```
str.endsWith("lo");
```

9. **indexOf(char ch)** - Returns the index of the first occurrence of the character.

Syntax –

```
str.indexOf('a');
```

10. **replace(char oldChar, char newChar)** - Replaces all occurrences of oldChar with newChar.

Example –

```
package Basic_Concept;
```

```
public class stringdemo {
```

```
    public static void main(String[] args) {
```

```
        String name="FortuneCloud";
```

```
        System.out.println("String is: "+name);
```

```
        System.out.println("Length of the String is: "+name.length());

        System.out.println("Uppercase: "+name.toUpperCase());

        System.out.println("Lowercase: "+name.toLowerCase());

        System.out.println("Equals: "+name.equals("FortuneCloud"));

        System.out.println("Char At: "+name.charAt(5));

        System.out.println("Index of: "+name.indexOf("o"));

        System.out.println("Ends with: "+name.endsWith("d"));

        System.out.println("Starts with: "+name.startsWith("h"));

        System.out.println("Replace: "+name.replace("e","a"));

        System.out.println("Concat: "+name.concat("Technology"));

    }

}
```

Output –

String is: FortuneCloud

Length of the String is: 12

Uppercase: FORTUNECLLOUD

Lowercase: fortunecloud

Equals: false

Char At: n

Index of:1

Ends with: true

Starts with: false

Replace: FortunaCloud

Concat: FortuneCloudTechnology